UNIVERSITY COMMUNICATION OFFICE · TASK MANAGEMENT SYSTEM

UCO-TMS Conceptual Model

How a request becomes a tracked, owned, and fulfilled service — the entities, routing rules, roles, and lifecycles behind the system.

9 domain tables | 10 services | 8 roles | 4 lifecycles



USE ← → TO NAVIGATE · F FOR FULLSCREEN

SYSTEM AT A GLANCE

One request model, four operational layers.

Intake, storage, service routing, and account state are separated — so every team sees only the work it owns.

9 tables
Domain entities
two are standalone request tables

10 services
Public and internal
routed across four owning teams

8 roles
Account types
two privileged roles bypass checks

4 lifecycles
Status families
integer codes stored on each row

A small schema with explicit routing beats a table per service.

01 **Intake**
Public forms and team dashboards create a Requestor parent.

02 **Request records**
One child row per request, linked by a shared request_id.

03 **Services**
Discriminators — specification and category — route the work.

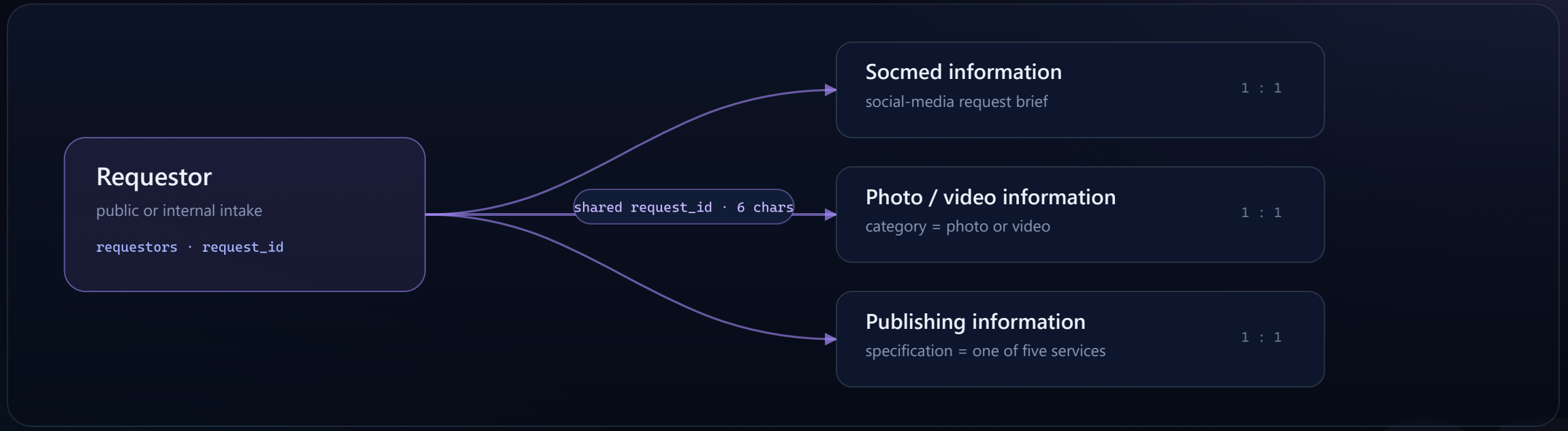
04 **Accounts & state**
Roles, employee_id assignment, per-account notifications.

Admin and superadmin can open every layer; team roles are scoped to their own dashboards.

INTAKE

One parent. One child. One shared key.

Every content or creative submission creates a **Requestor** parent and exactly one child request row. Both carry the same 6-character `request_id`.



Public landing forms
Social media, layout, production, trademark, soft copy.

Team dashboards
Photo, video, and creative work also enter from inside the system.

Standalone exception
Soft copy and trademark keep their own record — no Requestor parent.

REQUEST RECORDS

One table can serve many services.

Discriminators keep the schema compact: specification splits creative work, category splits media, and standalone requests keep their own tables.

`publishing_request_informaitons`

Five creative services

One row per service request. The **specification** column selects the dashboard.

Layout | Production | Publishing | UZ Post | UZ PR

Dashboard filter: `where specification = …`

`photo_vid_request_informations`

Two media services

The **category** column separates photography from videography on the same brief.

Photography | Videography

Event, venue, time, purpose, equipment — one shared shape.

STANDALONE

Soft copy request

Own request_id · no Requestor parent · yearbook team

STANDALONE

Trademark request

Own request_id · no Requestor parent · admin team

SERVICE CATALOGUE

Ten services, four owning teams.

Each service maps to one record shape and one team dashboard. Ownership is enforced by role middleware, not by convention.

CONTENT TEAM 03

- **SM** Social media — socmed table
- **PH** Photography — photo / video table
- **VD** Videography — category = video

CREATIVE TEAM 05

- **LY** Layout — specification = layout
- **PR** Production — specification = production
- **PB** Publishing — specification = publishing
- **UP** UZ Post — specification = uzpost
- **UR** UZ PR — specification = uzpr

YEARBOOK TEAM 01

- **SC** Soft copy — standalone table

year level · program | photo date | ID attachment

ADMIN 01

- **TM** Trademark — standalone table

trademark to use | date to use | category · audience

Why it stays small: five creative services share one table; two media services share one table; only the standalone workflows get their own records.

ASSIGNMENT & NOTIFICATIONS

Work is assigned by employee ID — not account ID.

The child row's `produce` column stores the assignee's employee_id. Notification feeds are derived; read receipts are per account.

Assignment path

One key joins staff accounts to request rows.



Assignment is a string match on `employee_id`. It is not a numeric foreign key to `users.id`.

Notification state

The feed is derived; the receipt is stored.

USER_ID	NOTIFICATION_KEY	READ_AT
12	socmed:AB12CD	—
12	layout:XY98ZQ	10:42
18	trademark:41	—

unique (user_id, notification_key)

Unread simply means no receipt row exists for this account and this key. One user opening a request never clears the badge for anyone else.

LIFECYCLES

Four pipelines, one integer per row.

Status codes live on the request row. Terminal states feed archives; team dashboards filter on them.

Content
socmed · photo / video

0 Pending → 1 In progress → 2 Posted / done

Creative
publishing table · five specs

0 Pending → 1 Approved → 2 In progress → 4 Sent

branch 3 Denied

Soft copy
standalone · yearbook

0 Pending → 1 Sent / done

branch 2 Denied

Trademark
standalone · admin

0 Pending → 1 Approved / sent

branch 2 Denied

Creative note: status 1 is reused for Approved and for Mark-as-done; `date_approved` records when approval happened.

ROLES & ACCESS

Eight roles, scoped to the work.

Team roles open only their own dashboards. Admin and superadmin pass every role check.

`content_team`
Content Team
Social media, photography, video

`content_admin`
Content Supervisor
Content dashboards + photo export

`creative_team`
Creative Team
Layout, production, publishing, UZ

`creative_admin`
Creative Supervisor
Creative dashboards + archive exports

`yearbook_team`
Yearbook Team
Soft-copy dashboard

`user`
Viewer
Authenticated; no team dashboard

`admin`
Admin
Trademark, users, every dashboard

`superadmin`
Super Admin
Unrestricted; set directly in DB

01 Public intake needs no account.

02 Team dashboards are protected by role middleware.

03 Admin and superadmin are always allowed.

04 Exports are restricted: content_admin for photos, creative_admin for creative archives.

05 Notification types are filtered by the same role rules.

admin and **superadmin** bypass every role check — including dashboards, exports, and notifications.

TAKEAWAYS

What the model makes possible.

A compact schema, explicit routing, and account-level state — built to stay understandable as the service catalogue grows.

01
One intake contract
A parent and one child row share a 6-character request_id.

02
Compact service routing
specification and category replace a table per service.

03
Clear ownership
Role-gated dashboards and employee_id assignment.

04
Account-level state
Derived notifications, per-account read receipts.



UCO-TMS · Conceptual model · 2026